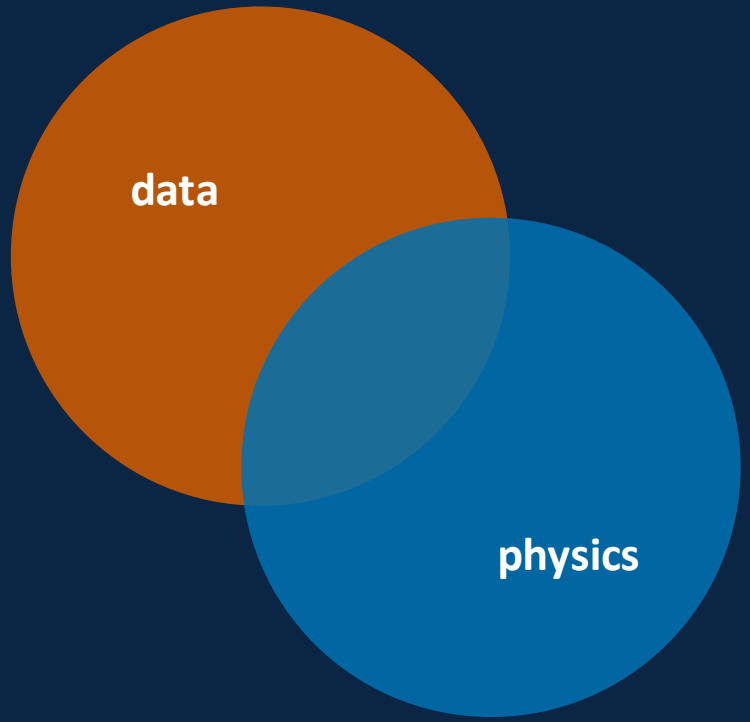
Physics-Informed AI for Environmental Data Science



HIDSI Workshop · Friday, October 16, 2026

Frederike Monte
PhD Candidate, Atmospheric Sciences · University of Hawai‘i at Mānoa

Workshop Overview

~25 min

Concepts

- What is physics-informed AI?
- How to add physics to a neural network
- Examples from Environmental Data Science

~45 min

Follow-along notebook

- Emulate humidity from weather-station data
- Train a neural network with and without physics
- Compare and visualize the results

~15 min

Discussion & wrap-up

- When do constraints help?
- Limitations
- Questions & survey

What Is Physics-Informed AI?

PHYSICS-INFORMED AI · THE UMBRELLA TERM

Using prior knowledge from our observational, empirical, physical or mathematical understanding of the world to improve a learning algorithm.

after Karniadakis et al. (2021)

PINNs · THE APPROACH WE USE TODAY

"Neural networks that are trained to solve supervised learning tasks while respecting any given laws of physics described by general nonlinear partial differential equations."
— Raissi et al. (2019)

Why do we need it?

- **Physically inconsistent predictions:** data-driven models can fit observations well yet produce physically implausible outputs *(Karniadakis et al., 2021)*
- **Large data needs, limited generalization:** ML needs lots of high-quality data and may fail outside its training range, e.g., in a changing climate *(Kashinath et al., 2021)*

Goals of physics-informed ML

Kashinath et al. (2021)

- Physically consistent predictions
- Data efficiency — fewer training data
- Faster training convergence
- Better generalization to unseen scenarios
- More transparent and interpretable

Also known as:
- Knowledge-guided ML (KGML)
- Part of scientific ML (SciML)

Kashinath et al. (2021); Markowski (2025)

How Does It Work? Three Ways to Add Physics

Making a model physics-informed = introducing biases that steer learning toward physically consistent solutions *(Karniadakis et al., 2021)*

Observational bias

through the data

Train on data that embody the physics, or augment the data (e.g., add rotated or shifted samples).

+ Simple — no change to the model
– Needs lots of data; physics not guaranteed

Inductive bias

through the architecture · hard constraint

Design the network so it satisfies physical laws by construction (e.g., conservation layers, symmetries).

+ Physics strictly satisfied
– Harder to design; limited to simple, known constraints

TODAY

Learning bias

through the loss function · soft constraint

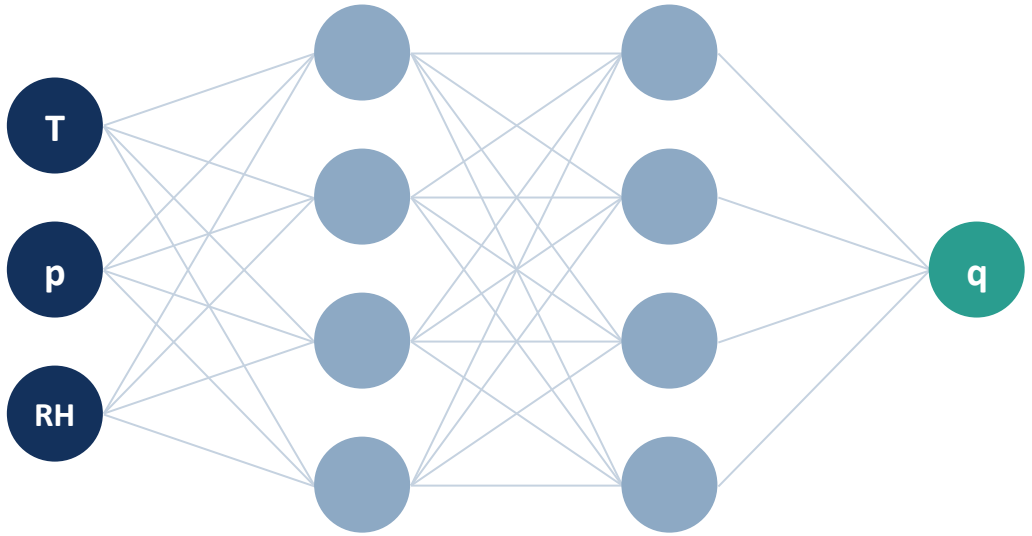
Add a penalty for violating physical laws during training (e.g., PINNs).

+ Flexible: works with any (differential) equation
– Physics only approximately satisfied

Not mutually exclusive: the three approaches can be combined *(Karniadakis et al., 2021)*

Hybrid models: a physics-based model with ML components, e.g., NeuralGCM *(Markowski, 2025; Kochkov et al., 2024)*

Data-Driven Neural Network



Each layer: $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$

Network: $\hat{y} = f(\mathbf{x})$

x: inputs (T, p, RH)

h: output of a layer → next layer

W, b: weights & biases (learned)

σ: activation function, e.g., tanh

ŷ: prediction (here: q)

Training loop

1. **Forward pass**
   Inputs flow through the layers → prediction ŷ
2. **Loss calculation**
   Mean squared error between ŷ and observations y:
   $$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$
3. **Backpropagation**
   Gradients ∂L/∂W, ∂L/∂b via automatic differentiation
4. **Weight update**
   Gradient descent with learning rate η:
   $$\mathbf{W} \leftarrow \mathbf{W} - \eta\frac{\partial L}{\partial \mathbf{W}}$$

1 epoch = one pass through all training data; repeat for many epochs

Adding Physics to the Loss Function

1 Physics = differential equations

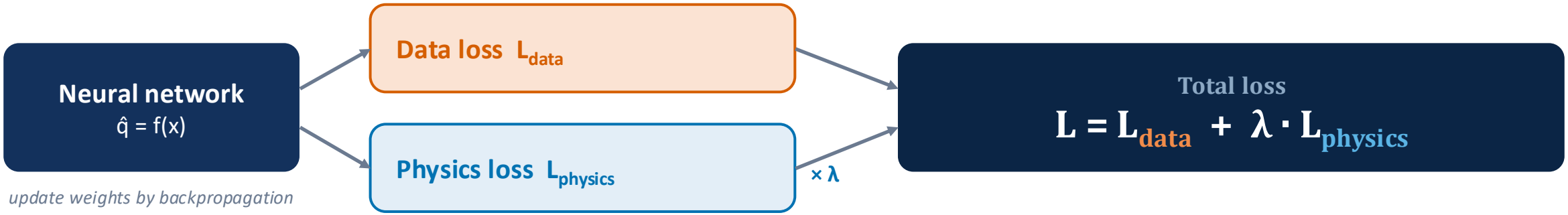
Most laws that govern a system's dynamics can be written as F(q) = 0, e.g., conservation laws or Clausius–Clapeyron

2 NNs are differentiable

Automatic differentiation gives exact derivatives of the output, e.g., ∂q̂/∂T, at any input — the same tool as backpropagation

3 Residual → extra loss term

Plug the derivatives into the equation; whatever doesn't add up (the residual) is penalized in the loss



L_{data}

Fit the observations: compare q̂ with measured q at the training points

$L_{physics}$

Penalize violating physics: the residual F(q̂) at collocation points — needs only inputs, no measured q, so it works even where there are no data

λ

Sets the trade-off between data and physics. Physics is encouraged, not guaranteed (soft constraint)

PINNs: Raissi et al. (2019); Karniadakis et al. (2021)

Comparison of Modeling Approaches

Physics-based models	Physics-informed AI	Purely data-driven ML
+ Grounded in physical laws	✓ Learns from data	+ Fast predictions
+ Interpretable, generalize well	✓ Respects known physics	+ Learns complex patterns from data
– Computationally expensive	✓ Generalizes better, needs less data	– Needs lots of data
– Not available when the physics is poorly understood	✓ Can be fast once trained	– Can violate physics, extrapolates poorly

Goal of physics-informed AI: combine the strengths of both → models that learn from data but stay physically consistent.

Based on Markowski (2025); Kashinath et al. (2021); Karniadakis et al. (2021)

Applications in Environmental Science

Atmosphere

- **Turbulent convection:** a GAN emulator is penalized when its fields don't match the covariance of the training data — more realistic and up to 80% faster to train *(Wu et al., 2020)*
- **Downscaling:** a GAN sharpens coarse wind fields 4× and reproduces the small-scale energy spectrum *(Singh et al., 2019)*
- **Hybrid weather & climate models:** NeuralGCM combines learned physics with a differentiable dynamical core *(Kochkov et al., 2024)*
- **Cloud microphysics (ongoing):** NNs trained on high-resolution simulations are embedded in a differentiable atmospheric model and refined with ARM and satellite observations *(Argonne PRECISION project)*

Ocean

- **North Atlantic circulation:** training labels come from an interpretable transform of the governing equations, so predictions trace back to physical regimes *(Sonnewald & Lguensat, 2021)*

Hydrology

- **Lake temperature:** an LSTM is pretrained on process-model output and penalized for violating energy conservation *(Read et al., 2019)*
- **Rainfall–runoff:** a mass-conserving LSTM keeps track of all water in and out by design *(Hoedt et al., 2021)*

Remote sensing & radiative transfer

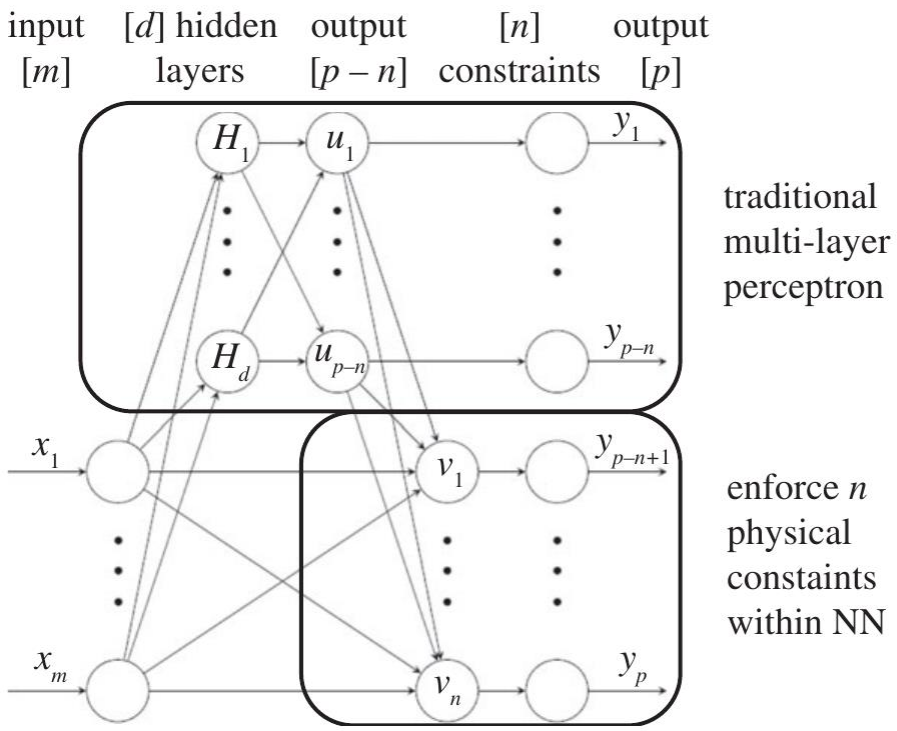
- **PINNs for radiative transfer:** the residual of the radiative transfer equation is the loss; solves forward and inverse problems *(Mishra & Molinaro, 2021)*
- **Atmospheric correction:** a physics-based simulator, tuned by autodiff, infers atmospheric transmission from a spectral scene *(Koch et al., 2024)*

Beyond environmental science

- **Fluid dynamics:** velocity and pressure fields inferred from flow visualizations *(Raissi et al., 2020)*
- **Medicine:** arterial blood pressure predicted from non-invasive 4D-flow MRI data *(Kissas et al., 2020)*

GAN: generative adversarial network · LSTM: long short-term memory · GCM: general circulation model

Example 1: Conserving Energy in Climate Model Emulators



Hard-constrained NN: a standard multi-layer perceptron (top) maps the inputs x to most outputs (u). The conservation layers (bottom) compute the remaining n outputs (v) directly from the conservation equations, so the constraints are satisfied exactly. Reproduced from Kashinath et al. (2021), Fig. 5; originally Beucler et al. (2019).

PROBLEM & GOAL

Convection and clouds are smaller than a climate model's grid cells, so they have to be parameterized (approximated). Goal: a neural network that predicts how each layer is heated and moistened from the large-scale state.

PHYSICS

The emulator should conserve mass, energy (enthalpy) and radiation, just like the climate model it replaces.

APPROACH

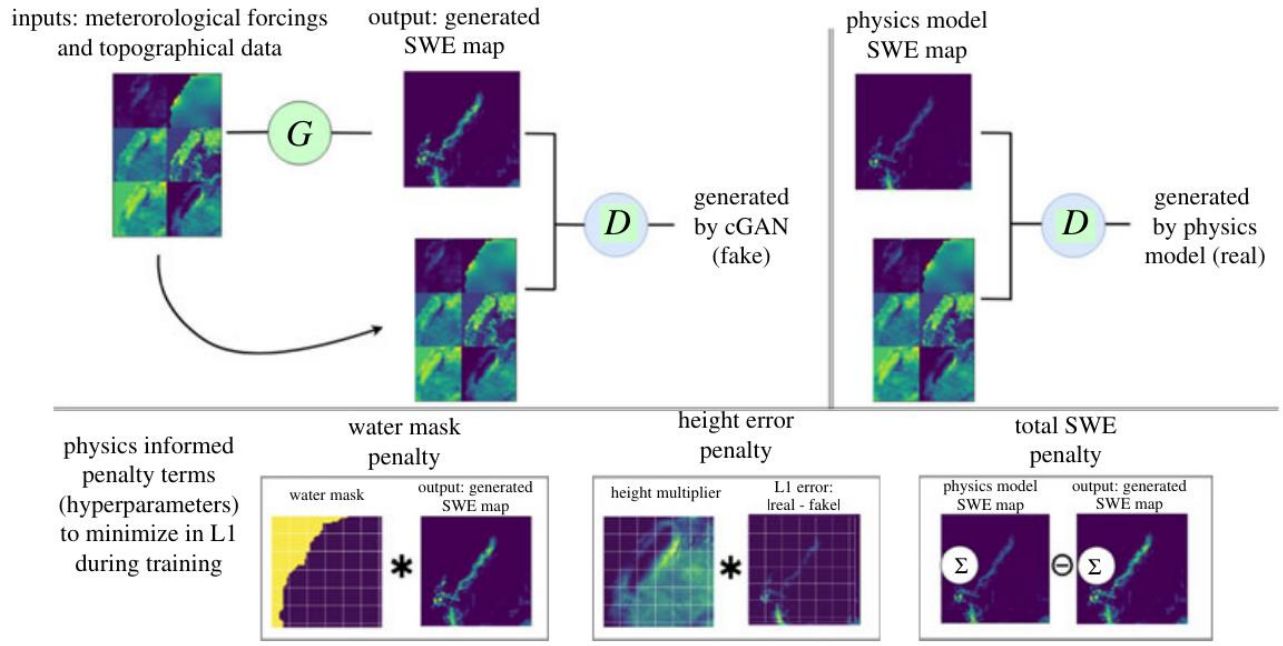
Three networks compared: unconstrained; a **soft constraint** (conservation penalty in the loss); and a **hard constraint** (conservation layers built into the architecture, see figure).

RESULTS

- **Unconstrained:** low error, but strongly violates conservation
- **Soft (loss):** far fewer violations
- **Hard (architecture):** conservation to machine precision
- Both constrained NNs generalize better to a +4 K warmer climate

Beucler et al. (2019, 2021); reviewed in Kashinath et al. (2021)

Example 2: Emulating Hydroclimate Models with cGANs



Top: the generator G turns meteorological and topographic input maps into a SWE map; the discriminator D judges whether a map is generated (fake) or comes from the physics model (real). Bottom: the three physics-informed penalty terms (water mask, height error, total SWE) added to the loss. Reproduced from Kashinath et al. (2021), Fig. 3 (top).

PROBLEM & GOAL

Physics-based hydroclimate models are slow to run. Goal: a fast emulator that turns gridded weather and topography maps into maps of snow water equivalent (SWE), the water stored in the snowpack.

APPROACH: DOMAIN KNOWLEDGE IN THE LOSS

A conditional GAN: the generator G creates SWE maps; the discriminator D tries to tell them apart from the physics model's maps. Three penalty terms are added to the loss:

- **Water mask:** no snow over water bodies
- **Height error:** errors at high elevation, where most snow is, count more
- **Total SWE:** total snow water must match the physics model

RESULTS

- **250× faster** than the numerical model
- Domain knowledge matters: removing the height-error term increased the error (RMSE) by 40 %

Manepalli et al. (2019); reviewed in Kashinath et al. (2021)

Our Example: Emulating Specific Humidity

Task: predict specific humidity q from temperature T, pressure p and relative humidity RH

Inputs: T, p, RH → **Neural network:** 2 × 32 neurons → **Output:** q

Jena Climate weather-station data (2009–2016, hourly): train on cool conditions, test on warm conditions the model has never seen

Training data: T < 20 °C | **Test: T ≥ 20 °C (extrapolation)**

Model 1: data only

$L = L_{data}$

Learns the q–T relation purely from observations

Model 2: data + physics

$L = L_{data} + \lambda \cdot L_{physics}$

Also penalized when predictions violate Clausius–Clapeyron (next slides)

Background: Calculating Specific Humidity

1 **Saturation vapor pressure** $e_s(T)$ [hPa]

How much water vapor can the air hold? The maximum at temperature T; it depends only on T and grows roughly exponentially (Clausius–Clapeyron, next slide)

2 **Actual vapor pressure** e [hPa]

$$e = RH/100 \times e_s$$

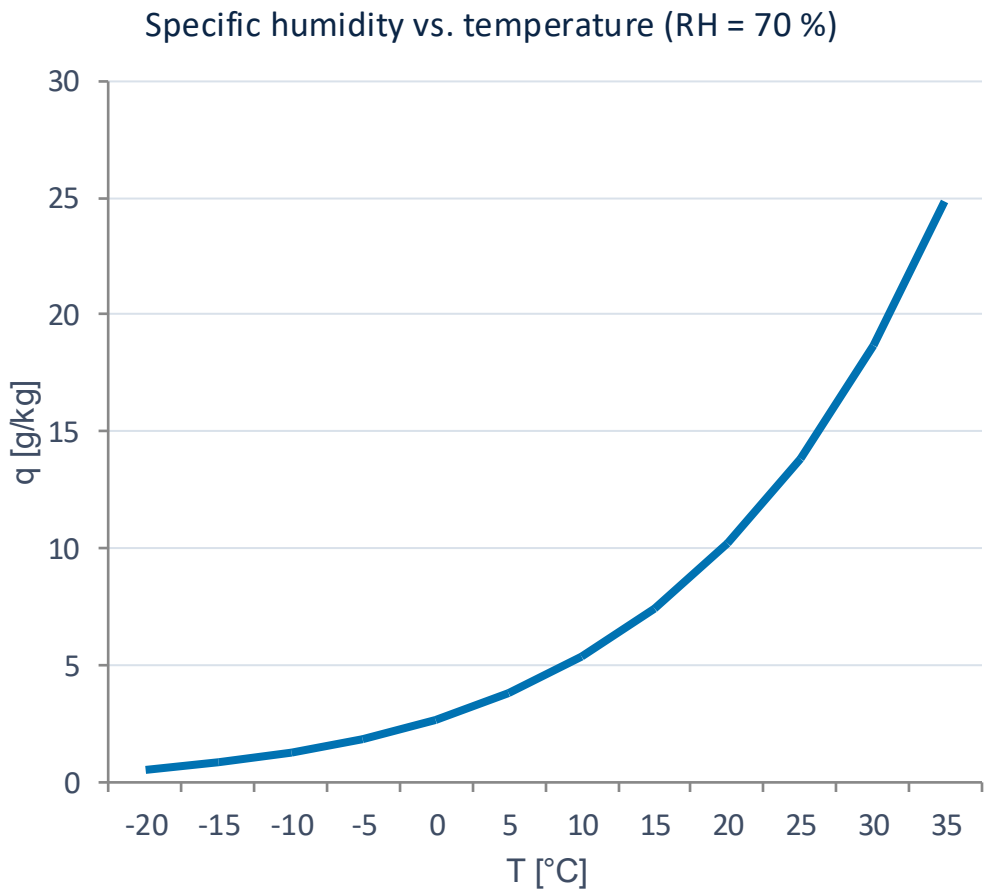
How much does it actually hold? RH [%] = how close the air is to saturation

3 **Specific humidity** q [g/kg]

$$q = 1000 \cdot \frac{0.622\, e}{p - 0.378\, e}$$

How much water is in each kg of air? grams of water vapor per kg of moist air

- **p:** air pressure [hPa]
- **0.622:** ratio of the molar masses of water vapor and dry air (≈ 18/29)
- **0.378 = 1 – 0.622:** small correction, since water vapor is part of the total pressure → q ≈ 622 e/p
- **× 1000:** converts kg/kg to g/kg



Full formulas (Magnus formula for e_s) in the notebook, section 1

Background: Clausius–Clapeyron

Clausius–Clapeyron

Describes how fast the saturation vapor pressure e_s grows when the temperature rises:

$$\frac{d \ln e_s}{dT} = \frac{L_v}{R_v T^2}$$

in words: the relative increase of e_s per kelvin

L_v: latent heat of vaporization ≈ 2.5×10^6 J/kg

R_v: gas constant of water vapor = 461.5 J/(kg K)

T: temperature in kelvin

≈ 6–7 % more water vapor per 1 K of warming at typical temperatures (≈ 8.5 % at −20 °C, ≈ 5.7 % at 35 °C)

Same law as the integrated form $e_s = e_0 \exp[(Lv/Rv)(1/T_0 - 1/T)]$. Differentiating its logarithm gives the equation above

What we add to Model 2

At fixed RH and p, q is proportional to e_s → q grows at the same relative rate:

$$\frac{\partial q}{\partial T} = \frac{L_v}{R_v T^2} q$$

The relationship between q and T (not a single number): how fast q must grow depends on the temperature and on how much water vapor is already there

How we use it: collocation points

- Every training step, we draw 512 random inputs (T, p, RH) with T from −20 to 35 °C: the **collocation points**
- They have **no measured q**: we only check whether the network's prediction follows this relationship (∂q/∂T via automatic differentiation)
- Violations are penalized in the physics loss, also from 20 to 35 °C, where there are **no training data**

Approximate physics: Lv held constant; small correction terms neglected



Any questions so far?

Next: follow-along notebook

Follow-Along Notebook: Let's Get Started

1

Go to the GitHub repository

Scan the QR code or type the link

2

Open the notebook

Clone the repo and run it locally or use Google Colab (click the badge in the README)

3

Run the "Setup check" cell

It should print package versions without errors

4

Follow along

Run each cell as we go — and try changing the settings in the “Try it yourself” cells



github.com/FrederikeM9/
HIDSI_Physics-Informed_AI

Discussion

What we found

RMSE (root-mean-square error): the typical size of the prediction error in g/kg — lower is better

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{q}_i - q_i)^2}$$

q̂: prediction, q: observation, N: number of test samples

	Cold test	Warm test
Model 1: data only	0.014	0.280
A: positive + monotonic	0.013	0.315
B: + convex	0.042	0.389
Model 2: Clausius–Clapeyron	0.037	**0.136**

Only the correct growth rate (Clausius–Clapeyron) halves the warm-range error, direction (A) or shape (B) alone don't help

What this tells us

1

Physics guides the model where there are no data

The physics loss needs no labels. Checked at collocation points up to 35 °C, it tells the network how fast q must grow beyond the training data.

2

Approximate physics costs a little accuracy

Our Clausius–Clapeyron constraint is approximate (constant Lv, neglected terms), so data and physics disagree slightly: a small trade-off on the cold test, controlled by λ.

3

Constraints must target the model's errors

Model 1 never went negative or downhill, it just grew too slowly. A was already satisfied; B made training harder without fixing the growth rate.

Limitations of Physics-Informed AI

In general: when it may not help

any physics-informed approach

- **Not one-size-fits-all:** sometimes traditional methods still perform better *(Markowski, 2025)*
- **Lots of data, little known physics:** a purely data-driven approach may be most effective *(Karniadakis et al., 2021)*
- **Constraints must target the model's errors:** in our notebook, positivity and monotonicity didn't help, the model already satisfied them

Specific to PINNs: training challenges

physics in the loss function

- **Physics only approximately satisfied:** data and physics losses compete; the physics loss is rarely exactly zero
- **Hard to optimize:** the PDE term makes the loss landscape ill-conditioned → vanilla PINNs can fail (~100% error) even for simple convection problems *(Krishnapriyan et al., 2021)*
- **Choosing λ is tricky:** too small and the physics is ignored; too large and the data fit and training suffer
- **Multiscale problems & cost:** spectral bias against high-frequency features; derivatives at many collocation points are expensive *(Karniadakis et al., 2021)*

Remedies for PINNs: curriculum regularization, sequence-to-sequence training, hard constraints *(Krishnapriyan et al., 2021)*

Key Takeaways

1

Three ways to add physics

Through the data, the architecture, or the loss function → and they can be combined

2

Physics helps most where data are scarce

Biggest gains when extrapolating, e.g., to a +4 K climate (Beucler) or our warm test range

3

Soft vs. hard constraints

A physics term in the loss is flexible but only approximately satisfied; architectural constraints are exact but harder to design

4

Physics has to fit the problem

PINNs can be hard to train, and constraints only help if the physics is right and targets the model's errors

Karniadakis et al. (2021); Kashinath et al. (2021); Krishnapriyan et al. (2021); Markowski (2025)

Thank you!

Questions? Please fill out the workshop survey.

Workshop materials: github.com/FrederikeM9/HIDSI_Physics-Informed_AI
Contact: *fmonte@hawaii.edu*



Survey

https://go.hawaii.edu/zuR

References

Main references

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. Nature Reviews Physics, 3, 422–440. https://doi.org/10.1038/s42254-021-00314-5

Kashinath, K., Mustafa, M., Albert, A., Wu, J.-L., Jiang, C., Esmaeilzadeh, S., Azizzadenesheli, K., Wang, R., Chattopadhyay, A., Singh, A., Manepalli, A., Chirila, D., Yu, R., Walters, R., White, B., Xiao, H., Tchelepi, H. A., Marcus, P., Anandkumar, A., Hassanzadeh, P., & Prabhat (2021). Physics-informed machine learning: case studies for weather and climate modelling. Phil. Trans. R. Soc. A, 379(2194), 20200093. https://doi.org/10.1098/rsta.2020.0093

Limitations

Krishnapriyan, A. S., Gholami, A., Zhe, S., Kirby, R. M., & Mahoney, M. W. (2021). Characterizing possible failure modes in physics-informed neural networks. In Proceedings of the 35th International Conference on Neural Information Processing Systems (NeurIPS '21), Article 2033, 26548–26560. Curran Associates Inc.

Online resources

Brunton, S. Physics Informed Neural Networks (PINNs) [Physics Informed Machine Learning]. YouTube. https://www.youtube.com/watch?v=-zrY7P2dVC4

Markowski, M. (2025, June 23). What is physics-informed machine learning? MathWorks Deep Learning Blog. https://blogs.mathworks.com/deep-learning/2025/06/23/what-is-physics-informed-machine-learning/

Moseley, B. (2021). So, what is a physics-informed neural network? https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/

Physics-Informed Neural Networks (PINNs). Kaggle notebook. https://www.kaggle.com/code/newtonbaba12345/physics-informed-neural-networks-pinns

Data

Jena Climate dataset: Weather Station Beutenberg, Max Planck Institute for Biogeochemistry, Jena. https://www.bgc-jena.mpg.de/wetter/

Kaggle download: https://www.kaggle.com/datasets/mnassrib/jena-climate

References

Atmospheric examples

Beucler, T., Rasp, S., Pritchard, M., & Gentine, P. (2019). Achieving conservation of energy in neural network emulators for climate modeling. arXiv:1906.06622. https://doi.org/10.48550/arXiv.1906.06622

Beucler, T., Pritchard, M., Rasp, S., Ott, J., Baldi, P., & Gentine, P. (2021). Enforcing analytic constraints in neural networks emulating physical systems. Physical Review Letters, 126, 098302. https://doi.org/10.1103/PhysRevLett.126.098302

Kochkov, D., Yuval, J., Langmore, I., Norgaard, P., Smith, J., Mooers, G., Klöwer, M., Lottes, J., Rasp, S., Düben, P., Hatfield, S., Battaglia, P., Sanchez-Gonzalez, A., Willson, M., Brenner, M. P., & Hoyer, S. (2024). Neural general circulation models for weather and climate. Nature, 632, 1060–1066. https://doi.org/10.1038/s41586-024-07744-y

Manepalli, A., Albert, A., Rhoades, A., & Feldman, D. (2019). Emulating numeric hydroclimate models with physics-informed cGANs. NeurIPS 2019 Workshop on Tackling Climate Change with Machine Learning. https://www.climatechange.ai/papers/neurips2019/39

Singh, A., Albert, A., & White, B. (2019). Downscaling numerical weather models with GANs. Climate Change AI Workshop at the 33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.

Wu, J.-L., Kashinath, K., Albert, A., Chirila, D., Prabhat, & Xiao, H. (2020). Enforcing statistical constraints in generative adversarial networks for modeling chaotic dynamical systems. Journal of Computational Physics, 406, 109209. https://doi.org/10.1016/j.jcp.2019.109209

Argonne National Laboratory. Physics-informed machine learning of cloud microphysics for high-resolution Earth system modeling with observationally constrained online training (PRECISION). https://www.anl.gov/genesis-mission/projects/physics-Informed-machine-learning-of-cloud-microphysics-for-high-resolution-earth-system-modeling-with-observationally-constrained-online-training

References

Hydrology, ocean & remote sensing

Hoedt, P.-J., Kratzert, F., Klotz, D., Halmich, C., Holzleitner, M., Nearing, G., Hochreiter, S., & Klambauer, G. (2021). MC-LSTM: Mass-conserving LSTM. Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR 139.

Koch, J., Forland, B., Bernacki, B., Doster, T., & Emerson, T. (2024). Data-driven invertible neural surrogates of atmospheric transmission. IGARSS 2024 – IEEE International Geoscience and Remote Sensing Symposium, Athens, Greece, 6943–6947. https://doi.org/10.1109/IGARSS53475.2024.10642124

Mishra, S., & Molinaro, R. (2021). Physics informed neural networks for simulating radiative transfer. Journal of Quantitative Spectroscopy and Radiative Transfer, 270, 107705. https://doi.org/10.1016/j.jqsrt.2021.107705

Hydrology, ocean & remote sensing (cont.)

Read, J. S., Jia, X., Willard, J., Appling, A. P., Zwart, J. A., Oliver, S. K., Karpatne, A., Hansen, G. J. A., Hanson, P. C., Watkins, W., Steinbach, M., & Kumar, V. (2019). Process-guided deep learning predictions of lake water temperature. Water Resources Research, 55(11), 9173–9190. https://doi.org/10.1029/2019WR024922

Sonnewald, M., & Lguensat, R. (2021). Revealing the impact of global heating on North Atlantic circulation using transparent machine learning. Journal of Advances in Modeling Earth Systems, 13(8), e2021MS002496. https://doi.org/10.1029/2021MS002496

Beyond environmental science

Kissas, G., Yang, Y., Hwuang, E., Witschey, W. R., Detre, J. A., & Perdikaris, P. (2020). Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering, 358, 112623. https://doi.org/10.1016/j.cma.2019.112623

Raissi, M., Yazdani, A., & Karniadakis, G. E. (2020). Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. Science, 367, 1026–1030. https://doi.org/10.1126/science.aaw4741